NEOLABS

SECURITY LABS · SOC LEVEL 1

SOC Level 1 Guided Lab Pack

Synthetic defensive investigations and analyst command references

Version: week1-prod-232

Publication date: 2026-10-28

Classification: Student Training Material

Authorised synthetic training use only.

Contents

1. Practice Lab 01 — Authentication Failure Chain

2. NeoLabs SOC Level 1 Query and Command Reference

Source: labs/01-authentication-triage/README.md

Practice Lab 01 — Authentication Failure Chain

Track: SOC Level 1
Difficulty: Beginner → intermediate
Estimated time: 60–90 minutes
Dataset: `sample-logs/authentication/failed-login-chain.ndjson`
Scope: Synthetic `pod-03` training records only

Scenario

A Wazuh alert reports repeated authentication failures involving a service account. A successful login appears later in the same period, but one telemetry source also reports a collection gap.

Your task is to perform a defensible Level 1 triage. Do not assume that the alert proves compromise, and do not ignore the visibility gap.

Learning objectives

You should be able to:

- distinguish normal and suspicious authentication sequences;
- pivot by account, source address, session and correlation identifiers;
- build an event-time timeline;
- identify what the available records prove and what remains uncertain;
- recognise a cross-pod access-control denial without attempting to reproduce it;
- document a telemetry gap and explain its effect on confidence;
- prepare an incident report, evidence log and escalation package.

Safety boundary

- Use only the supplied synthetic dataset or the approved local Wazuh stack.
- Do not send requests to any VCC pod, endpoint or public address as part of this practice lab.
- Reserved example IP addresses in the dataset are documentation values, not targets.
- Do not attempt to change pod scope or access another pod.

Preparation

Study:

1. `docs/secops-foundations/01-security-operations-foundations.md`
2. `docs/secops-foundations/02-alert-triage-evidence-and-escalation.md`
3. `docs/secops-foundations/03-siem-pipelines-and-log-quality.md`
4. The Log Literacy manual sections on time, correlation and evidence statements

Copy these templates into your working folder:

- `templates/incident-report-template.md`
- `templates/evidence-log-template.md`
- `templates/query-journal-template.md`

Part A — Validate the dataset

Before analysing the security activity:

1. Confirm that every non-empty line is valid JSON.
2. Count the records.
3. Identify the schema version and pod identifier.
4. Confirm that each event is marked `synthetic: true`.
5. Compare `event_time` with `ingest_time` for delayed records.
6. Note any event type that represents telemetry health rather than user activity.

Example local validation command:

```
python3 - <<'PY'
import json
from pathlib import Path

path = Path("sample-logs/authentication/failed-login-chain.ndjson")
records = [json.loads(line) for line in path.read_text().splitlines() if line.strip()]
print(f"records={len(records)}")
print(f"pods={sorted({record['pod_id'] for record in records})}")
print(f"schemas={sorted({record['schema_version'] for record in records})}")
print(f"all_synthetic={all(record.get('synthetic') is True for record in records)}")
PY
```

Record this command and result in your query journal.

Part B — Authentication pivots

Answer the following with evidence IDs and exact queries:

1. Which accounts experienced authentication failures?
2. Which source addresses generated failures?
3. How many failures involved `svc-backup`?

4. Did any successful authentication follow those failures?
5. Which session was created by the relevant success?
6. Did the same source target additional accounts?
7. Which later events reference the suspicious session?
8. Is there a normal failure-followed-by-success sequence elsewhere in the dataset? How does its context differ?

You may use Python, `jq`, Wazuh dashboard filters or another approved local tool. Record the exact method.

Part C — Timeline

Build a timeline beginning five minutes before the first relevant failure and ending five minutes after the session revocation.

Your timeline should include:

- authentication failures;
- successful authentication;
- application activity;
- authorization decisions;
- account changes;
- session action;
- telemetry-health events.

Use **event time** for the primary order and note meaningful ingest delay separately.

Part D — Evidence reasoning

Write an evidence statement for each of the following:

1. Repeated failures for the service account.
2. Successful login from the same source.
3. Activity performed through the new session.
4. The denied cross-pod request.
5. The sensitive account change.
6. The process-telemetry gap.

For each statement, include:

- what the record directly proves;
- what it suggests in context;
- what it does not prove;
- the next source that would reduce uncertainty.

Part E — Classification

Choose one current classification:

- false positive;
- benign positive;
- suspicious;
- confirmed synthetic incident;
- insufficient evidence;
- data-quality issue.

Then provide:

- confidence: low, medium or high;
- potential severity;
- the strongest supporting facts;
- alternative explanations considered;
- the effect of the telemetry gap;
- whether the case should be escalated.

The quality of your reasoning matters more than selecting a label without explanation.

Part F — Deliverables

Submit the following through the programme's approved assignment repository when this lab is formally assigned:

```
lab-01-authentication-triage/
├── incident-report.md
├── evidence-log.md
├── query-journal.md
└── screenshots/
    └── README.md
```

The screenshots folder may contain approved Wazuh views, but do not include credentials, private URLs, unrelated alerts or another pod's data.

Review checklist

- [ ] Facts are separated from interpretation.
- [ ] Every important claim cites an evidence ID.
- [ ] Queries include their time ranges and data source.
- [ ] A zero-result search is not treated as proof without checking source health.
- [ ] The cross-pod denial is documented, not reproduced.
- [ ] The telemetry gap is included in the confidence assessment.
- [ ] No secret or private endpoint appears in the submission.
- [ ] The final recommendation stays within SOC Level 1 authority.

Instructor separation

This public repository intentionally contains no hidden answer key or scenario ground truth. Mentor review material belongs in a separate private repository.

Source: references/query-command-reference/README.md

NeoLabs SOC Level 1 Query and Command Reference

Version: 0.2-draft
Review baseline: 1 August 2026
Test scope: Synthetic files, the learner's local Wazuh stack and explicitly authorised VCC data only

> A query is a question expressed in a platform's syntax. Record the exact query, data source, time field and time range so another analyst can reproduce the result.

1. Choose the correct language

Interface or source	Primary syntax	Typical use
Wazuh specialised tabs and server API	Wazuh Query Language (WQL)	Filter agents, rules, decoders, groups and supported API resources
Wazuh/OpenSearch Discover	Dashboard search syntax and field filters	Search indexed alert or archive documents
Wazuh Indexer Dev Tools	OpenSearch Query DSL	Structured Boolean, range, aggregation and exact-field queries
NDJSON/JSON files	`jq` or Python	Validate and analyse synthetic datasets
Plain-text logs	`grep`, `awk`, `sed` with care	Fast local inspection and counting
Linux system journal	`journalctl`	Filter systemd journal by time, unit, priority and fields
Linux audit logs	`ausearch`	Search audit records by event type, key, user, time and success
Windows event logs	PowerShell `Get-WinEvent`	Filter channels, IDs, time windows and structured event properties

Do not mix query languages. A WQL expression is not automatically valid OpenSearch DSL, PowerShell or shell syntax.

Part I — Wazuh Query Language

2. WQL structure

WQL uses:

```
field operator value
```

Operators:

```
=   equal
!=  not equal
>   greater than
<   less than
~   like/matching text
```

Separators:

```
;   AND
,   OR
()  grouping
```

Examples:

```
status=active
status=active;os.name=Ubuntu
status=active,(os.name=Ubuntu,os.name=Debian)
name~web
```

Use double quotes around values containing spaces:

```
name="Windows Server 2025"
```

WQL is case-sensitive. Use the field suggestions and validation provided by the dashboard tab.

3. WQL cautions

- Field availability depends on the selected tab or API endpoint.
- `;` means AND and `,` means OR; this differs from many programming languages.
- The `~` operator is not a regular-expression guarantee. Confirm its behaviour in the specific endpoint.

- When using the API, URL-encode reserved characters or use a client option that performs encoding.
- WQL filters management/API data; indexed security-event searches may use another syntax.

4. WQL investigation record

```
Question: Which active agents report Ubuntu?
Interface: Agents management / supported WQL search
Query: status=active;os.name=Ubuntu
Time run: 2026-08-01 10:00 UTC
Result: [record count and summary]
Limitation: Agent status does not prove current event delivery for every configured source.
```

Part II — Dashboard field filters and Discover

5. Build searches from fields

In Discover or Threat Hunting:

1. select the correct data view, such as `wazuh-alerts-*`;
2. set an absolute time range;
3. inspect an event to confirm field names;
4. add include/exclude filters from field values;
5. display only useful columns;
6. sort by the relevant event timestamp;
7. record every inherited filter.

Useful VCC fields include:

```
pod_id
event_type
action
outcome
user
source_ip
session_id
correlation_id
event_id
schema_version
synthetic
```

Field paths can be nested or prefixed after Wazuh indexing. Inspect the actual event rather than assuming the path.

6. Exact versus analysed fields

Search engines may store a text field in two forms:

- analysed text, useful for word searching;
- keyword/exact value, useful for equality, grouping and aggregation.

If `user` searches return unexpected partial matches, inspect whether an exact/keyword form exists. Do not invent a `.keyword` suffix without checking the mapping.

7. Time-range discipline

Record:

- start and end in UTC;
- the dashboard time field;
- browser time-zone setting;
- whether the source also has `event_time` or `ingest_time`;
- any known delay or clock skew.

A relative range such as “last 15 minutes” is not reproducible later.

Part III — OpenSearch Query DSL

8. Basic exact query

Use Dev Tools only in an authorised local environment and only against approved indices.

```
GET wazuh-alerts-*/_search
{
  "size": 50,
  "query": {
    "term": {
      "data.pod_id": "pod-03"
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

A `term` query expects an exact value and is normally appropriate for keyword-like fields. Confirm the mapping first.

9. Boolean filter

```
GET wazuh-alerts-*/_search
{
  "size": 100,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.event_type": "authentication" } },
        { "term": { "data.user": "svc-backup" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

Use `filter` for exact constraints that do not need relevance scoring.

10. Search one source across accounts

```
GET wazuh-alerts-*/_search
{
  "size": 100,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.source_ip": "203.0.113.44" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "sort": [
    { "timestamp": "asc" }
  ]
}
```

11. Count by account

```
GET wazuh-alerts-*/_search
{
  "size": 0,
  "query": {
    "bool": {
      "filter": [
        { "term": { "data.pod_id": "pod-03" } },
        { "term": { "data.event_type": "authentication" } },
        { "term": { "data.outcome": "failure" } },
        {
          "range": {
            "timestamp": {
              "gte": "2026-08-01T09:00:00Z",
              "lte": "2026-08-01T09:30:00Z"
            }
          }
        }
      ]
    }
  },
  "aggs": {
    "failures_by_user": {
      "terms": {
        "field": "data.user",
        "size": 20
      }
    }
  }
}
```

If the aggregation fails, verify that the field is aggregatable and mapped as an exact/keyword value.

12. Check field mappings

```
GET wazuh-alerts-*/_mapping/field/data.user
```

Use mappings to confirm type and exact-field behaviour. Do not change production mappings from a student lab.

13. Query DSL cautions

- `match` performs analysed full-text search and is not the same as `term`.
- Wildcards at the beginning of a value can be expensive.
- Large `size` values can overload the browser or indexer.
- Aggregation counts can be misleading when duplicate events exist.
- The `_source` document may contain sensitive fields; export only what the assignment requires.
- A successful query proves only that matching indexed documents were found.

Part IV — jq for NDJSON and JSON

14. Validate every NDJSON line

```
jq -c . sample-logs/authentication/failed-login-chain.ndjson >/dev/null
```

No output and exit code `0` means each non-empty line was valid JSON. It does not validate the event schema.

15. Display selected fields

```
jq -c '{event_time,event_id,event_type,user,source_ip,outcome,session_id}' \
  sample-logs/authentication/failed-login-chain.ndjson
```

16. Filter authentication failures

```
jq -c 'select(.event_type == "authentication" and .outcome == "failure")' \
  sample-logs/authentication/failed-login-chain.ndjson
```

17. Filter by account and sort by time

For NDJSON, first read the stream into an array:

```
jq -s '
  map(select(.user == "svc-backup"))
  | sort_by(.event_time)
  | .[]
  | {event_time,event_id,action,outcome,source_ip,session_id}
' sample-logs/authentication/failed-login-chain.ndjson
```

Use `-s` carefully with large files because it loads all records into memory.

18. Count failures by user

```
jq -s '
  map(select(.event_type == "authentication" and .outcome == "failure"))
  | group_by(.user)
  | map({user: .[0].user, count: length})
' sample-logs/authentication/failed-login-chain.ndjson
```

`group_by` expects sorted input by the grouping expression. jq sorts as part of `group_by`, but review output rather than assuming every missing user field behaves as intended.

19. Find duplicate event IDs

```
jq -r '.event_id' sample-logs/authentication/failed-login-chain.ndjson \
  | sort \
  | uniq -d
```

No output means no duplicate IDs were found in the examined file. It does not prove upstream collectors never duplicated semantically identical events under different IDs.

20. Check pod and synthetic markers

```
jq -s '{
  pods: (map(.pod_id) | unique),
  schemas: (map(.schema_version) | unique),
  all_synthetic: all(.[]; .synthetic == true)
}' sample-logs/authentication/failed-login-chain.ndjson
```

Part V — grep, awk and text logs

21. Fixed-string search

Prefer fixed-string mode when you do not need regular expressions:

```
grep -F 'Failed password' sample.log
```

22. Include line numbers and context

```
grep -n -C 2 -F '203.0.113.44' sample.log
```

23. Count matching lines

```
grep -c -F 'authentication failure' sample.log
```

Line count is not automatically event count. Multiline events and duplicate records can change the meaning.

24. Safe recursive search

```
grep -RIn --exclude='*.key' --exclude='.env' -F 'correlation-id' ./approved-log-folder
```

Run recursive searches only inside the authorised local working directory. Avoid accidentally scanning credential folders or unrelated personal files.

25. awk field example

For a controlled space-delimited sample:

```
awk '$9 == 401 {print $1, $4, $7, $9}' access.log
```

Real web log fields can contain quoted spaces and custom formats. `awk` column assumptions must be verified against the exact log format.

Part VI — journalctl

26. Query by unit and time

```
journalctl --unit=sshd.service \
  --since='2026-08-01 09:00:00 UTC' \
  --until='2026-08-01 10:00:00 UTC' \
  --no-pager
```

27. Query by priority

```
journalctl --priority=warning..alert --since='1 hour ago' --no-pager
```

28. Output JSON

```
journalctl --unit=sshd.service --since='today' --output=json --no-pager \
  | jq -c '{time:.__REALTIME_TIMESTAMP,unit:._SYSTEMD_UNIT,pid:._PID,message:.MESSAGE}'
```

`__REALTIME_TIMESTAMP` is normally expressed in microseconds since the Unix epoch. Convert carefully before mixing it with ISO 8601 timestamps.

29. Filter structured journal fields

```
journalctl _SYSTEMD_UNIT=ssh.service _COMM=sshd --since='today' --no-pager
```

Unit names differ between distributions. Confirm whether the system uses `ssh.service` or `sshd.service`.

30. journalctl cautions

- Access may require elevated local permissions.
- Volatile journals can disappear after reboot.

- Retention depends on journal configuration and storage.
- Service messages are application text inside a structured journal; parse them cautiously.
- A missing record may mean the unit did not log it, not that the action did not happen.

Part VII — ausearch

31. Failed login records

```
ausearch --message USER_LOGIN --success no --start today --interpret
```

32. Search by audit key

```
ausearch --key identity_changes --start today --interpret
```

The audit key exists only when an audit rule was configured with that key.

33. Unsuccessful system calls

```
ausearch --message SYSCALL --success no --start today --interpret
```

This can produce high volume. Narrow by time, user, executable or audit key where appropriate.

34. ausearch cautions

- Audit records for one action can span multiple related messages with the same audit event identifier.
- `--interpret` improves readability but may resolve numeric values using current system state.
- Audit rules must exist before the activity occurs.
- Use elevated permissions only on a system you own or are authorised to administer.

Part VIII — PowerShell and Windows Event Logs

35. List recent failed logons

```
$start = [datetime]'2026-08-01T09:00:00Z'
$end   = [datetime]'2026-08-01T10:00:00Z'

Get-WinEvent -FilterHashtable @{
    LogName   = 'Security'
    Id        = 4625
    StartTime = $start.ToLocalTime()
    EndTime   = $end.ToLocalTime()
} | Select-Object TimeCreated, Id, MachineName, RecordId
```

`Get-WinEvent` uses local `DateTime` values for the filter. Record how UTC times were converted.

36. Inspect structured event properties

```
$event = Get-WinEvent -FilterHashtable @{
    LogName = 'Security'
    Id      = 4625
} -MaxEvents 1

[xml]$xml = $event.ToXml()
$xml.Event.EventData.Data | ForEach-Object {
    [pscustomobject]@{
        Name  = $_.Name
        Value = $_.'#text'
    }
}
```

Property positions can differ between event versions. Named XML fields are safer than relying only on numeric property indexes.

37. Sysmon process creation

```
Get-WinEvent -FilterHashtable @{
    LogName   = 'Microsoft-Windows-Sysmon/Operational'
    Id        = 1
    StartTime = (Get-Date).AddHours(-1)
} | Select-Object TimeCreated, Id, RecordId, Message
```

The message is convenient for reading, but structured XML fields such as `ProcessGuid`, `Image`, `CommandLine`, `ParentProcessGuid` and hashes are stronger for correlation.

38. Export selected events

```
Get-WinEvent -FilterHashtable @{
    LogName = 'Microsoft-Windows-Sysmon/Operational'
    Id      = 1,3,22
} -MaxEvents 200 |
    Export-Csv -Path '.\approved-output\sysmon-review.csv' -NoTypeInformation
```

Before sharing, review exported command lines, user names, paths and addresses for sensitive information.

39. Windows event cautions

- Event 4624 proves a logon session was created, not that it was authorised.
- Event 4625 proves a logon attempt failed, not why the attempt occurred.
- Event 4688 command-line visibility depends on audit policy.
- Sysmon coverage depends on its installed version and configuration.
- Event messages can be localised; structured fields are more portable.
- Clearing or overwriting a log creates visibility limitations that must be documented.

Part IX — Query-writing method

40. Ten questions before drawing a conclusion

1. Which source produced the record?
2. Which time field am I using?
3. Which identity acted or was targeted?
4. Which asset, object or service was affected?
5. What action and outcome are recorded?
6. Which fields are original, decoded or enriched?
7. What preceding and following events matter?
8. What normal behaviour or maintenance context exists?
9. What does this result fail to prove?
10. Which next query would reduce uncertainty most?

41. Query journal standard

Every important query should record:

```
Question being tested:
Platform and source:
Exact query:
Time field:
Absolute UTC range:
Result count:
Important result:
Negative finding:
Evidence IDs:
Limitations:
Next pivot:
```

42. Common query mistakes

- using free text when an exact structured field exists;
- searching ingest time while interpreting the result as event time;
- leaving a hidden dashboard filter active;
- using `match` when exact equality is required;
- grouping duplicate events without checking IDs;
- treating zero results as proof before validating the source;
- pasting secrets or private URLs into commands, screenshots or reports;
- running commands outside the approved local or VCC scope;
- copying a query from another SIEM without translating field names and semantics.

Authoritative references

- Wazuh, *Filtering data using Wazuh Query Language*.
- Wazuh, *Wazuh indexer indices* and *Event logging*.
- OpenSearch, *Query DSL* and mapping documentation.
- jq manual.
- GNU grep and GNU awk manuals.
- systemd `journalctl` manual.
- Red Hat audit and `ausearch` documentation.
- Microsoft Learn, `Get-WinEvent`, Windows auditing and Sysmon documentation.
- `research/AUTHORITATIVE_SOURCE_REGISTER.md`.